

Lesson 4.3 — Reviewing AI Output in Front of a Panel

Lesson 4.3 — Reviewing AI Output in Front of a Panel

The third live format: a panel opens a PR on the shared screen and says “review this.” What they are testing is whether you have an *AI-aware* reviewing checklist that catches things their juniors miss.

By the end of this lesson:

- You will have a senior-grade AI code review checklist with eleven items.
 - You will know the three categories of comment that signal seniority on a PR.
 - You will know how to deliver review feedback in front of an audience without sounding harsh.
 - You will have a worked example walking a panel through a PR review.
-

1. Why this format exists

Reviewing AI-generated code in 2026 is its own discipline. The tells of bad AI output differ from the tells of bad human output. Companies who use AI heavily have learned the hard way that “average reviewers” miss AI-specific failure modes. They want to hire people who don’t.

The format usually opens like this:

“Here is a PR one of our junior engineers submitted. They used AI assistance. Review it as if you’re the senior on the team. Walk us through your thinking.”

The PR is salted with one or two real issues. **Your job: find them, narrate, and demonstrate the review discipline.**

2. The eleven-item AI review checklist

Print this. Reference it during practice. Internalize it before any panel.

2.1. Does it do what was asked?

The most basic question. Read the PR description. Read the diff. Are they about the same thing? AI loves to do “more than asked”; juniors accept the bonus work.

2.2. Does every API call point to a real, current API?

Hover over imports. Check that methods exist. AI fabricates plausibly-named methods. **A 30-second sanity check on imports catches this most of the time.**

2.3. Does it respect the codebase’s existing patterns?

Find a sibling file with similar responsibilities. Compare. AI imports its training data’s style; your codebase has its own. Mismatch = future maintenance pain.

2.4. Are there any one-character bugs hiding in plain sight?

Off-by-one, wrong comparison operator, missing await, accidentally-async function. AI produces these at a slightly higher rate than careful humans.

2.5. Were tests added that *test the new behavior*, not just *pass*?

Open the test file. Read each test. Does it actually exercise the change? Common AI failure: tests that import the new code and assert on a trivial property, giving false confidence.

2.6. Are there empty try/catch blocks?

AI loves swallowing errors. Search for catch in the diff. Every empty body is a flag.

2.7. Are there any types?

In a TypeScript codebase, any is a smell. AI sometimes reaches for any instead of working out the correct type. Each one needs justification.

2.8. Are there hardcoded values that should be parameters or constants?

Magic numbers, hardcoded strings, baked-in URLs. AI defaults to plausible values rather than asking. Each one is a refactor invitation.

2.9. Was anything *deleted* that should not have been?

Read the diff with an eye on the minus signs. AI is capable of helpfully removing lines it deems unnecessary that turn out to be load-bearing.

2.10. Does the code match the PR description?

If the description says “fix bug X” and the diff also adds an unrelated feature, that is two changes in one PR. Push back.

2.11. Could you ship this if you owned it?

The final gut-check. Imagine the PR is yours. Imagine being on call when it breaks. Would you ship it as-is? If no, articulate why.

An eleven-item checklist sounds like a lot. With practice, you can run it on a 50-line diff in under 3 minutes.

3. The three categories of senior review comments

Not all review comments carry the same signal. Three kinds carry the most.

3.1. The “this won’t compose” comment

“This is fine in isolation, but useUserScores is going to be called inside <TeamRow>, which itself is rendered in a list of 50. That means 50 hook invocations per render. We should hoist this up.”

You named the second-order effect. **High-leverage signal.**

3.2. The “this hides a bug we’ll see in production” comment

“The empty catch on line 47 will swallow the network error. We will not see the failure in our metrics. Either rethrow or log.”

You traced an immediate flaw to a runtime consequence. **Direct senior signal.**

3.3. The “future maintainer will struggle” comment

“This name `useThing` will not survive a year. In six months someone will `grep` for the actual feature and miss this entirely. Suggest `useTeamScoreSubscription` or similar.”

You thought about a person who is not in the room. **The most senior signal of all.**

4. The three categories of *non-senior* comments to avoid

4.1. The taste comment without justification

“I’d use destructuring here.”

Why? The interviewer cannot tell if you have a reason. Add a sentence.

4.2. The drive-by nitpick

“Missing semicolon.”

Either Prettier handles it (so it does not matter) or it does not (so the team has a deeper config issue). Either way, the comment by itself is a low-signal use of attention.

4.3. The “rewrite the whole thing” comment

“I would refactor this whole module.”

You just told the panel you do not respect scope boundaries. Even if the module *should* be refactored, **a PR review is not the place to propose it.**

5. The verbal script for live review

Your narration during a live review follows this rhythm.

5.1. The opening

“OK, let me start with the high-level – read the PR description, then walk the diff. I’ll call out things as I see them.”

Pace yourself. Reading is allowed. Silence for 30 seconds while you read is fine.

5.2. The structural pass

After reading, narrate the structural impressions:

“Three things at the structural level. One, the diff touches four files which matches the description, so scope is OK. Two, the test file changes are isolated to one new test – let me come back to whether that test actually covers the change. Three, no dependency changes. Good. Now the line-by-line.”

That is roughly 20 seconds of high-altitude commentary that sets up the rest of the review.

5.3. The line-by-line pass

For each meaningful diff hunk:

- **Read the change in your head.**
- **State what it does in your own words.**
- **Identify any concern.** If none, say “no concern.”
- **Move on.**

“Line 47, this is the new helper. It filters by status, sorts by date, slices to the limit. No concerns... wait. The sort is comparing dates as strings. ISO strings sort correctly lexicographically, which works only if all dates are full ISO format. Let me check what’s calling this...”

The “wait” moment is gold. Interviewers see the live discovery. **Do not edit your reactions.**

5.4. The summary pass

After the diff:

“OK, top-line: I would request changes on three things. One, the empty catch on line 92 – needs a logger. Two, the hardcoded 8 in the helper – should be a parameter. Three, I want to see a test that actually covers the new path, not just the existing one. The rest looks good. Want me to draft the comments?”

That summary lands the review. **It mirrors what an offer-track candidate would say in real life.**

6. The worked example

Pretend the PR description says:

Description: Add a helper to surface the top 3 most recent verified users.

Pretend the diff is:

```
// lib/users.ts
export function getTopRecent(users: any[]): any[] {
  try {
    return users
      .filter(u => u.verified)
      .sort((a, b) => a.signupAt - b.signupAt)
      .slice(0, 3);
  } catch (e) {
    return [];
  }
}

// lib/users.test.ts
test('returns an array', () => {
  expect(Array.isArray(getTopRecent([]))).toBe(true);
});
```

Walk the panel through this.

Structural pass

"Two files in the diff: the implementation and one test. Description matches scope. No new dependencies. Good baseline."

Line-by-line concerns

"First, the parameter is typed `any[]` and the return is `any[]`. We have a `User` type in `Lib/types.ts` – this should use that. So that is checklist item 7, the `any` type concern.

Second, the sort comparator is `a.signupAt - b.signupAt`. The diff treats `signupAt` as a number, but our `User` type has it as an ISO string. Subtracting strings produces NaN. The sort will silently not sort. That's a bug.

Third, the sort direction. The description says 'most recent', which means descending. The current sort, even if it worked, is ascending. So a second bug – after we fix the type issue, we need `b - a`, not `a - b`.

Fourth, the empty try/catch on the outside. It swallows everything. If there's a real error, we silently return empty and the user sees no top-3. That's checklist item 6.

Fifth, the hardcoded 3 in the slice. The description does say 'top 3' but a parameter would future-proof this. Checklist item 8. Lower priority – would mention but not block.

Sixth, the test. It only asserts that an array is returned for an empty input. It does not exercise the new behavior at all. We need at least three tests: returns 3 verified users sorted newest-first; returns fewer than 3 when only fewer qualify; returns empty when nothing qualifies. Checklist item 5."

Summary

"To summarize: I'd block this PR on three things – the type, the comparator (which has two bugs), and the missing real tests. I'd note the empty catch and the hardcoded 3 as non-blocking but should-fix. Three blockers, two suggestions."

That review:

- Hits seven of the eleven checklist items.
- Names two real bugs (type, comparator).
- Distinguishes blockers from suggestions.
- Closes with a count.

Eight minutes of speech. Memorable. Drill the structure on your own examples.

7. The body language and tone of live review

Three principles.

7.1. Critique the code, not the coder

The PR allegedly came from “a junior engineer.” **Do not say things like “this is sloppy” or “the AI clearly made a mess.”** Talk about the code in third-person, neutrally.

Bad: “Whoever wrote this didn’t think about...”

Good: “This pattern doesn’t handle the case where...”

7.2. Compliment what works

When something is correct, say so. *“The scope is appropriately small — that’s good.”* It signals that you read for both wins and losses, not just losses.

7.3. Pace your delivery

A reviewer who barrels through eight comments in two minutes sounds harsh. A reviewer who takes a breath after each major comment, makes eye contact, and lets the panel acknowledge before moving on — that reviewer sounds senior.

8. The “what would you ask the author?” question

Often the panel will ask:

“What questions would you ask the author before approving?”

This is a chance to demonstrate **collaborative review**. Have a structured answer:

“Three questions. One, was the AI assistance generated from a single prompt or iterated? If iterated, what was the original ask – sometimes the best fix is to re-prompt with stronger constraints. Two, has this been tested in the real environment? The unit test passing is not the same as production-like data. Three, is there a related ticket – sometimes a PR is solving the wrong problem because the requirement upstream was off.”

That answer treats the author as a peer, asks about their process, and references upstream work. **Senior, generous, not pedantic.**

9. The “this PR was AI-generated” cultural question

A wildcard but increasingly common:

“How do you talk to a teammate who consistently submits AI-generated PRs you have to clean up?”

The HR-aware answer:

“Privately, kindly, and concretely. The conversation is not ‘stop using AI.’ It is ‘here are three patterns I’ve been catching in your PRs that suggest the AI is slipping past your review – let’s pair on the next one and I’ll show you the catches I make.’ Make it about review skill, not tool choice. Most people respond well; the few who do not need a manager involvement, but I always start at the peer level.”

That answer demonstrates:

- You can have hard conversations.
- You distinguish behavior from identity.
- You start with the kindest plausible interpretation.
- You know when to escalate.

Drill it.

10. The “AI-augmented review tools” question

“What’s your take on AI-powered code review tools? Greptile, CodeRabbit, GitHub’s review suggestions, etc.?”

Be specific:

“They’re useful for the bottom-of-the-funnel stuff – style, missing types, dead code, basic security smells. They miss the things that matter most: design choices, second-order effects, and code that conflicts with our team’s history. I treat them like a linter – green should be a precondition for human review, but it should never replace it. The risk is that teams treat green as approved and skip the human pass. That’s how the worst PRs sneak through.”

Notice the structure: useful, but here are the limits, here is how I integrate them, here is the risk to watch for. That is a senior, calibrated answer to any “what do you think of tool X?” question.

11. Practice exercises

11.1. The salted PR drill

Take a real, working PR from your repo (open-source or your own). Have a friend salt it with **one** of the AI failure-mode signatures from Lesson 4.2. Open it cold. Time yourself reviewing. Did you find the salt? How long did it take?

11.2. The peer review

Pair with another developer. Each of you submits a small AI-assisted PR to the other for review. Treat it as a real interview format – narrate aloud, written comments after. Switch roles. **Watch how each of you reviews differently.**

11.3. The recorded review

Open any open-source PR you do not have context for. Record yourself reviewing it for 10 minutes. Watch back. Note the gaps in your narration and the comments you made that lacked justification.

12. Common pitfalls

12.1. Reading silently

If the panel cannot hear you think, they will not credit you with thinking. **Default to narration.** Even *“Reading the diff...”* is enough to fill the air.

12.2. Diving into the deepest detail first

Some candidates immediately zoom into a regex or a comparator and start picking it apart. Resist. **The structural pass first** — *“is the scope right? Are the right files touched?”* — sets you up to be the reviewer the panel wants to hire.

12.3. Performing skepticism

Some candidates over-criticize because they think it makes them look senior. The opposite. **The most senior reviewer says “this part is fine” the most often** — they have calibrated the bar precisely.

12.4. Forgetting to summarize

You spent 10 minutes finding issues. The panel needs a 30-second wrap. **Always summarize at the end.** Count the blockers. Count the suggestions. Hand it back.

13. CHECK YOURSELF

- ☐ Can you list the eleven-item review checklist from memory?
 - ☐ Can you name the three categories of senior comment with an example each?
 - ☐ Can you name the three categories of non-senior comment to avoid?
 - ☐ Can you walk a panel through a small AI-generated PR with the structural-then-line-by-line-then-summary rhythm?
 - ☐ Can you handle the “AI-generated PRs from a teammate” cultural question without being dismissive?
-

14. Where you are now

You can review AI output in front of a panel with confidence. You have completed the practical-format chapter — the three formats most likely to show up in your interview.

Move on to [../chapter-05-system-design/README.md](#) — system design and architecture, where the question shifts from “show me you can code with AI” to “show me you can think with AI in the room.”